

Programmation mobile avec Android

Cours complet

1 Introduction à Android

Présentation d'Android

Android est un système d'exploitation mobile développé par Google, basé sur un noyau Linux. Né en 2004, racheté par Google en 2005, puis publié en 2007 avec la version 1.5, Android en est actuellement à sa version 8 (Oreo) sortie en octobre 2017.

Définition 1.1 : Système Android

Android est un système complet pour smartphones et tablettes comprenant : (1) un système d'exploitation Linux sous-jacent pour la gestion matérielle, (2) des API de programmation pour les interfaces utilisateur et outils, (3) des applications natives (navigateur, courrier, etc.).

Évolution et obsolescence

L'évolution d'Android est très rapide et volontaire. Les concepts fondamentaux (principes, organisation) restent stables, mais la syntaxe (méthodes, paramètres, classes, ressources) évolue constamment.

Attention

Ce que vous allez apprendre sera rapidement dépassé (environ 1 an) sur le plan syntaxique. L'autoformation permanente est indispensable dans ce domaine.

Composition d'une application

Une application Android est composée de trois éléments principaux :

- **Sources Java ou Kotlin** : compilés pour la machine virtuelle Dalvik (versions ≤ 4.4) ou ART (depuis version 5)

- **Ressources** : fichiers XML (interface, textes) et images PNG (icônes)
- **Manifeste** : description du contenu, autorisations, signature

2 Environnement de Développement

SDK Android et Android Studio

Le SDK (Software Development Kit) contient les bibliothèques et outils nécessaires au développement. Android Studio est l'environnement de développement intégré (IDE) officiel basé sur IntelliJ IDEA.

Composant	Description	Utilité
SDK Platform	Bibliothèques Java	Indispensable
SDK Tools	Gestionnaire SDK	Indispensable
Platform-tools	ADB, fastboot	Communication USB
System Images	Images émulateur	Tests virtuels
AVD Manager	Gestionnaire émulateurs	Création AVD

Structure d'un projet Android Studio

Structure du projet ARBORSCENCE

```

MonApplication/
├── app/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/
│   │   │   │   ├── com/exemple/monapp/
│   │   │   │   │   └── MainActivity.java
│   │   │   │   └── ← Code source Java/Kotlin
│   │   │   ├── res/
│   │   │   │   ├── layout/
│   │   │   │   │   └── ← Interfaces XML
│   │   │   │   ├── values/
│   │   │   │   │   └── ← Strings, colors, arrays
│   │   │   │   ├── drawable/
│   │   │   │   │   └── ← Images
│   │   │   │   └── AndroidManifest.xml
│   │   │   │       └── ← Configuration
│   │   └── test/
│   │       └── ← Tests
│   ├── build.gradle
│       └── ← Build app
└── build.gradle
    └── ← Build projet

```

Remarque

Le système de build Gradle est automatiquement configuré par Android Studio. Les fichiers build.gradle ne nécessitent généralement pas de modification manuelle pour les projets simples.

3 Applications et Activités

Le concept d'Activité

Une **activité** (Activity) est un composant fondamental d'Android représentant un écran unique avec une interface utilisateur. Chaque activité est définie par une classe Java héritant de AppCompatActivity.

Exemple de classe Activity JAVA

```

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}

```

Cycle de vie d'une activité

Le système Android gère les activités selon un cycle de vie précis avec des méthodes de callback appelées automatiquement lors des changements d'état.

États du cycle de vie

- **onCreate()** : Création de l'activité, initialisation

- **onStart()** : L'activité devient visible
- **onResume()** : L'activité est active et interactive
- **onPause()** : Perte de focus, sauvegarde données
- **onStop()** : L'activité n'est plus visible
- **onDestroy()** : Destruction de l'activité

Le fichier AndroidManifest.xml

Chaque activité doit être déclarée dans le manifeste de l'application. Ce fichier XML décrit la structure et les exigences de l'application.

Déclaration d'activités XML

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <application
    android:icon="@drawable/app_icon"
    android:label="@string/app_name">

    <activity
      android:name=".MainActivity"
      android:label="@string/app_name">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>

    <activity android:name=".EditActivity" />

  </application>
</manifest>
```

4

Création d'Interfaces Utilisateur

Principe des ressources Android

Les interfaces sont définies par des fichiers XML placés dans le répertoire `res/layout/`. Le SDK génère automatiquement une classe `R` permettant d'accéder à ces ressources depuis le code Java.

Définition 4.1 : Identifiants de ressources

R.layout.nom	:identifiant	d'un	layout	XML
R.id.nom	:identifiant		d'une	vue
R.string.nom	:chaîne		de	caractères
R.drawable.nom	:		image ou drawable	

Les Layouts principaux

LinearLayout

Dispose les vues en ligne, horizontalement ou verticalement selon l'attribut orientation.

LinearLayout vertical XML

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Nom" />

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Entrer le nom" />

</LinearLayout>
```

ConstraintLayout (recommandé)

Layout flexible permettant des positionnements complexes avec des contraintes relatives.

ConstraintLayout XML

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/btnLogin"
        android:layout_width="0dp"
        android:layout_height="48dp"
        android:text="Se connecter"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBottom_toBottomOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Dimensions et unités

Attribut	Valeurs possibles	Description
layout_width/height	match_parent	Occupe tout l'espace disponible

layout_width/height	wrap_content	Taille minimale selon contenu
layout_width/height	100dp	Taille fixe en density-independent pixels
layout_weight	0, 1, 2...	Pondération dans LinearLayout

Remarque importante

L'unité **dp** (density-independent pixel) garantit une taille apparente identique sur tous les écrans quelle que soit leur densité de pixels.

5 Gestion des Événements

Principe des écouteurs

La gestion des interactions utilisateur repose sur le pattern **Listener** (écouteur). Les vues génèrent des événements auxquels l'activité réagit via des méthodes de callback.

Méthode 1 : L'activité comme écouteur

L'activité implémente l'interface OnClickListener et se déclare elle-même comme écouteur.

```

Implémentation OnClickListener JAVA

public class MainActivity extends AppCompatActivity
    implements View.OnClickListener {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button btn = findViewById(R.id.btn_valider);
        btn.setOnClickListener(this);
    }

    @Override
    public void onClick(View v) {
        if (v.getId() == R.id.btn_valider) {
            // Traitement du clic
        }
    }
}

```

Méthode 2 : Classe anonyme

Création d'une instance anonyme d'écouteur directement lors de l'affectation.

Classe anonyme JAVA

```
Button btn = findViewById(R.id.btn_valider);
btn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // Action spécifique à ce bouton
        Toast.makeText(MainActivity.this,
            "Validation effectuée",
            Toast.LENGTH_SHORT).show();
    }
});
```

Accès aux vues depuis le code

La méthode `findViewById()` permet d'obtenir la référence Java d'une vue définie en XML.

Manipulation des vues JAVA

```
// Récupération des vues
EditText edtNom = findViewById(R.id.edt_nom);
TextView tvResultat = findViewById(R.id.tv_resultat);
Button btnValider = findViewById(R.id.btn_valider);

// Modification des propriétés
edtNom.setHint("Entrez votre nom");
tvResultat.setText("Bienvenue !");
tvResultat.setTextColor(Color.BLUE);
btnValider.setEnabled(false);
```

6 Navigation avec Intents

Principe des Intents

Un **Intent** est un objet décrivant une opération à effectuer : lancement d'une activité, diffusion d'un message, ou interaction avec une application externe.

Définition 6.1 : Types d'Intents

Intent explicite : spécifie la classe cible par son nom complet. Utilisé pour la navigation interne à l'application.

Intent implicite : spécifie une action générique (VIEW, SEND, DIAL). Le système choisit l'application appropriée.

Intent explicite

Lancement d'une Activity JAVA

```
// Création de l'Intent avec contexte et classe cible
Intent intent = new Intent(MainActivity.this,
                           DetailsActivity.class);

// Transmission de données (extras)
intent.putExtra("user_name", "Mohammed");
intent.putExtra("user_age", 25);
intent.putExtra("is_admin", true);

// Lancement de l'activité
startActivity(intent);
```

Récupération des données

Dans l'Activity destinataire JAVA

```
Intent intent = getIntent();

String nom = intent.getStringExtra("user_name");
int age = intent.getIntExtra("user_age", 0);
boolean isAdmin = intent.getBooleanExtra("is_admin", false);

// Valeur par défaut si la clé n'existe pas
```

Intent implicite - Actions système

Action	Constante	URI Type
Ouvrir site web	ACTION_VIEW	https://...
Faire un appel	ACTION_DIAL	tel:0612345678
Envoyer SMS	ACTION_SENDTO	smsto:0612345678
Envoyer email	ACTION_SENDTO	mailto:test@mail.com
Ouvrir carte	ACTION_VIEW	geo:0,0?q=Paris

Exemple : ouvrir un navigateur JAVA

```
String url = "https://www.android.com";  
Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse(url));  
startActivity(intent);
```

7 Affichage de Listes

Architecture Adapter-View

L'affichage de listes repose sur le pattern **Adapter** : un objet intermédiaire entre les données (modèle) et la vue d'affichage (ListView ou RecyclerView).

Rôle de l'Adapter

- **getCount()** : retourne le nombre d'éléments
- **getItem(position)** : retourne l'élément à la position donnée
- **getView(position, convertView, parent)** : crée ou recycle la vue d'affichage

ArrayAdapter simple

Utilisé pour des listes simples affichant une seule chaîne de caractères par élément.

Création et association JAVA

```
// Données  
ArrayList<String> noms = new ArrayList<>();  
noms.add("Mercure");  
noms.add("Vénus");  
noms.add("Terre");  
  
// Création de l'adaptateur  
ArrayAdapter<String> adapter = new ArrayAdapter<>(  
    this, // Contexte  
    android.R.layout.simple_list_item_1, // Layout item  
    android.R.id.text1, // ID TextView  
    noms // Données  
);  
  
// Association au ListView  
ListView listView = findViewById(R.id.list_view);  
listView.setAdapter(adapter);
```

Adaptateur personnalisé

Pour des affichages complexes (plusieurs informations par item, images), il faut créer un adaptateur spécifique.

Classe PlaneteAdapter JAVA

```
public class PlaneteAdapter extends ArrayAdapter<Planete> {
```

```
public PlaneteAdapter(Context context, List<Planete> planetes) {
    super(context, 0, planetes);
}

@Override
public View getView(int position, View convertView, ViewGroup parent) {
    // Réutilisation ou création de la vue
    PlaneteView view = (PlaneteView) convertView;
    if (view == null) {
        view = PlaneteView.create(parent.getContext());
    }

    // Affichage des données
    Planete planete = getItem(position);
    view.setItem(planete);

    return view;
}
```

Optimisation importante

Le paramètre `convertView` permet de réutiliser les vues qui deviennent invisibles lors du défilement (pattern ViewHolder). Cette technique est essentielle pour les performances des listes longues.

RecyclerView (moderne)

Composant plus performant que ListView, obligatoire pour les nouvelles applications.

```
Configuration RecyclerView JAVA

RecyclerView recyclerView = findViewById(R.id.recycler_view);

// Layout Manager (liste verticale)
recyclerView.setLayoutManager(new LinearLayoutManager(this));

// Adaptateur
recyclerView.setAdapter(new PlaneteAdapter(listePlanetes));

// Optimisation
recyclerView.setHasFixedSize(true);
```

8

Stockage de Données

Modes de stockage

Android propose plusieurs solutions de persistance selon les besoins :

Méthode	Type de données	Accès
Preferences	Clés-valeurs simples	Privé à l'app
Fichiers internes	Fichiers binaires/texte	Privé à l'app
SQLite	Base relationnelle	Privé à l'app
JSON	Objets sérialisés	Exportable

Base de données SQLite

Solution standard pour le stockage structuré local. Architecture en 4 classes :

Architecture SQLite

1. **DatabaseContract** : définit les noms de tables et colonnes (constantes)
2. **DatabaseHelper** : étend SQLiteOpenHelper, gère création et mise à jour
3. **Modèle (User, Planete...)** : classe POJO représentant une ligne
4. **Repository** : encapsule les opérations CRUD

DatabaseHelper exemple JAVA

```
public class DatabaseHelper extends SQLiteOpenHelper {
    private static final String DB_NAME = "UserDB.db";
    private static final int DB_VERSION = 1;

    private static final String SQL_CREATE =
        "CREATE TABLE users (" +
        "id INTEGER PRIMARY KEY AUTOINCREMENT," +
        "username TEXT," +
        "email TEXT)";

    public DatabaseHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }

    @Override
```

```

    public void onCreate(SQLiteDatabase db) {
        db.execSQL(SQL_CREATE);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldV, int newV) {
        db.execSQL("DROP TABLE IF EXISTS users");
        onCreate(db);
    }
}

```

Format JSON

JavaScript Object Notation, format texte léger pour l'échange de données structurées.

Manipulation JSON JAVA

```

// Création
JSONObject personne = new JSONObject();
personne.put("nom", "Salma");
personne.put("prenom", "Karimi");
personne.put("age", 21);
personne.put("actif", true);

// Conversion en chaîne
String jsonText = personne.toString();

// Lecture
JSONObject obj = new JSONObject(jsonText);
String nom = obj.getString("nom");
int age = obj.getInt("age");

```

9

Fragments

Concept de Fragment

Un **Fragment** est un composant d'interface réutilisable représentant un comportement ou une partie d'écran. Il peut être combiné avec d'autres fragments dans une même activité, permettant des interfaces adaptatives (tablettes vs téléphones).

Définition 9.1 : Fragment

Un fragment possède son propre cycle de vie, reçoit ses propres événements d'entrée, et peut être ajouté ou supprimé dynamiquement pendant l'exécution de l'activité hôte.

Création d'un Fragment

Classe Fragment minimale JAVA

```

public class InfosFragment extends Fragment {

```

```
// Constructeur vide obligatoire
public InfosFragment() {}

@Override
public View onCreateView(LayoutInflater inflater,
                        ViewGroup container,
                        Bundle savedInstanceState) {
    return inflater.inflate(
        R.layout.infos_fragment,
        container,
        false
    );
}
```

Intégration dans une activité

Méthode statique (déclaration XML)

Layout avec fragments XML

```
<LinearLayout
    android:orientation="horizontal">

    <fragment
        android:id="@+id/frag_liste"
        android:name="fr.iutlan.fragments.ListeFragment"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1" />

    <fragment
        android:id="@+id/frag_infos"
        android:name="fr.iutlan.fragments.InfosFragment"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="2" />

</LinearLayout>
```

Méthode dynamique (programmation)

Transaction de fragments JAVA

```
FragmentManager manager = getSupportFragmentManager();
FragmentTransaction transaction = manager.beginTransaction();

// Remplacer le contenu du FrameLayout
transaction.replace(R.id.container, new InfosFragment());
transaction.addToBackStack(null); // Empiler pour retour arrière
transaction.commit();
```

Communication Fragment-Activité

Le fragment communique avec l'activité hôte via une interface callback.

Interface de communication JAVA

```
public class ListeFragment extends Fragment {
```

```

public interface OnItemSelectedListener {
    void onItemSelected(Item item);
}

private OnItemSelectedListener listener;

@Override
public void onAttach(Context context) {
    super.onAttach(context);
    listener = (OnItemSelectedListener) context;
}
}

```

10 Préférences et Fichiers

SharedPreferences

Stockage de préférences utilisateur sous forme de couples clé-valeur persistant.

Lecture et écriture JAVA

```

// Récupération
SharedPreferences prefs = PreferenceManager
    .getDefaultSharedPreferences(getApplicationContext());

boolean online = prefs.getBoolean("prefs_online", true);
String hostname = prefs.getString("prefs_host", "localhost");
int nbMax = Integer.parseInt(prefs.getString("prefs_nbmax", "100"));

// Modification
SharedPreferences.Editor editor = prefs.edit();
editor.putBoolean("prefs_online", false);
editor.putString("prefs_host", "192.168.1.1");
editor.commit(); // ou apply() asynchrone

```

Fichiers internes

Stockage de fichiers texte ou binaires dans l'espace privé de l'application.

Flux d'écriture et lecture

Écriture : Données → BufferedWriter → OutputStreamWriter → FileOutputStream → Fichier

Lecture : Fichier → FileInputStream → InputStreamReader → BufferedReader → Données

Écriture dans fichier JAVA

```

// Mode MODE_PRIVATE écrase, MODE_APPEND ajoute
FileOutputStream fos = openFileOutput("data.txt", MODE_PRIVATE);
BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(fos));

bw.write("Ligne 1");
bw.newLine();

```

```
bw.write("Ligne 2");  
bw.close();
```